

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barengi

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [] Zanero [] Carminati [] Barengi
Have you done any challenges/homework? [] Yes [] No
Do you want to withdraw? [] Yes [] No
Priority Grading [] Yes, Motivation: [] No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 14 pages. Check that you have all of them.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red. No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

DISTANCE MODE ONLY

- On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
- We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
- We will evaluate only the uploaded **readable** documents.
- USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

A large agricultural greenhouse relies on a smart industrial automation system to control environmental factors critical to crop growth, such as irrigation, temperature, and ventilation. The system includes sensors (e.g., soil moisture, CO₂ levels, temperature) and actuators (e.g., water pumps, windows, fans), all communicating wirelessly over HTTP with a central controller. Each sensor authenticates with the central controller using a challenge-response mechanism: at each communication round, the central controller sends a random challenge, and the sensor replies with a digest (e.g., hash) of a shared password and the random data. The central controller verifies the digest to ensure message authenticity. Remote access to the central controller is provided through a web dashboard served over HTTPS or via a secure VPN. The central controller also includes a USB maintenance port accessible by personnel.

[1.5 points] 1. Identify and describe the 2 most relevant assets at risk in such a scenario, along with their associated threats, threat agents. All elements must be motivated.

Asset	Threat	Threat Agent
Crop Health and Productivity	Attackers could indirectly damage or reduce crop yields by tampering with automation inputs (e.g., spoofing moisture/temperature readings), causing long-term economic loss.	Competitors aiming to disrupt supply, or pranksters with wireless access and technical know-how.
Company Reputation	If attackers cause crop loss, environmental damage, or public data breaches, customers and partners may lose trust in the greenhouse operator.	Disgruntled former employees who know system weaknesses, or nearby attackers exploiting wireless vulnerabilities.

[1.5 points] 2. Identify 3 potential vulnerabilities in the company's operations, specifying how they can be exploited by threat agents and a possible countermeasure.

Vulnerability	Exploit	Countermeasure
Lack of authentication on sensor messages	A nearby attacker can spoof moisture data, causing the system to over-irrigate or under-irrigate, damaging crops.	Use HMAC-based message with a shared secret and a nonce + digital signature to verify authenticity.
USB maintenance port lacks access control	An insider or intruder could connect a USB device and inject malicious firmware or extract configuration files.	Require physical authentication (e.g., smart card) or firmware signing and disable USB by default.
Cleartext wireless communication over proprietary protocol	An attacker with RF tools can eavesdrop on or replay messages to alter actuator behavior (e.g., disable fans).	Encrypt wireless traffic using lightweight cryptographic protocols suitable for IoT (e.g., AES-CCM).

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] Consider the following, simplified, relay network. A group of servers, acting as message relays, set up TCP connections between any pair of them. TCP connections are employed in a single direction (say, server A to server B), and, to allow duplex communication, two TCP sessions are present between any given server pair. The servers establish the symmetric key for each TCP connection by having the other endpoint send its public encryption key over the wire, and using it to encrypt a uniformly randomly drawn string of 256 bits. The ciphertext is sent to the other endpoint where it is decrypted by means of the endpoint private decryption key, allowing the recipient to obtain confidentially the 256 bit string, which is then employed to encrypt all messages with AES, and compute a MAC tag on the messages themselves.

Justify why the approach provides message confidentiality, or, in case you can find a vulnerability, provide an exploit.

The system fails at providing confidentiality, as a result of the fact that the public keys are not authenticated or integrity-protected in any way. An active attacker simply impersonates all the endpoints she desires by generating asymmetric encryption keypairs and swapping the public keys with the legitimate ones at symmetric key establishment time.

[3 points] Consider a password storage scheme which has the target of making the system more resilient to the accidental disclosure of passwords. Admissible passwords are 14 alphanumeric upper-and-lowercase characters strings, which are split in two equal parts, each part is hashed with a public known salt (one per user account), and the two digests are stored on separate servers.

Describe the advantages of such a system to manage a password-based authentication scheme with respect to an attacker obtaining only one of the storages of the servers, and against one obtaining both, with respect to a canonical salted-password hashing scheme.

For the sake of argument, assume that the passwords are uniform draws of 14 alphanumeric characters.

The system is actively damaging the security of the password-based authentication scheme; indeed, splitting the passwords in two halves allows for a square-root reduction in the bruteforce space. Indeed, for the sake of quantifying assuming the usual alphanumeric character set with the English alphabet, the complexity of a bruteforce attack moves from 62^{14} (approx 2^{84}) to $2 \cdot 62^7$ (approx 2^{43}), making exhaustive search computationally feasible whereas it was not before. An attacker gaining access to one half of the database can perform an exhaustive password search on it separately.

Exercise 3 - Binary Vulnerabilities (6 points)

After the **high number of clarification requests** for the last exam, professors have decided to **release the code of the automatic corrector** for the malware exercise.

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4.
5. #define NAME_SIZE 32
6. #define MAX_NOTES 256
7.
8. typedef struct {
9.     char name[NAME_SIZE];
10.    char notes[MAX_NOTES];
11.    int grade;
12. } Student;
13.
14. Student student;
15.
16. void max_grade() {
17.     student.grade = 6;
18.     printf("Student: %s, Grade: %d\n", student.name, student.grade);
19.     exit(0);
20. }
21.
22. void init_student() {
23.     char student_name[NAME_SIZE];
24.     memset(&student, 0, sizeof(Student));
25.     printf("Enter the student's name: ");
26.     scanf("%s", student_name);
27.     snprintf(student.name, NAME_SIZE, "%s", student_name);
28.     student.name[NAME_SIZE - 1] = '\0';
29.     student.grade = -1;
30. }
31.
32. void set_notes() {
33.     if (student.notes[0] == '\0') {
34.         printf("Insert the notes: ");
35.         fgets(student.notes, MAX_NOTES, stdin);
36.         printf(student.notes);
37.     }
38. }
39.
40. int random_grade() { return rand() % 5; }
41.
42. int main() {
43.     init_student();
44.     printf("Welcome to the automatic corrector for the Malware exercise!!\n");
45.     if (student.grade == -1) {
46.         student.grade = random_grade();
47.     }
48.     set_notes();
49.     printf("Student: %s, Grade: %d\n", student.name, student.grade);
50.
51.     return 0;
52. }
```

The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that canary is the only mitigation enabled.

Note: The global variables are inside the **.BSS** section.

Man

fgets(char *s, int size, FILE *stream): reads in at most one less than size characters from stream and stores them into the buffer pointed to by s. Reading stops after an EOF or a newline. If a newline is read, it is stored into the buffer. A terminating null byte ('\0') is stored after the last character in the buffer.

exit(int n): The exit utility causes the shell to exit from its current execution environment with the exit status specified by the unsigned decimal integer n.

rand(): The function returns a pseudo-random integer in the range 0 to RAND_MAX inclusive.

[2 point] 1. Fill out the following table by reporting all the vulnerabilities you can spot in the code. If you believe there might be a vulnerability, specify for each vulnerability the **corresponding line/s of code** and specify if it is possible to exploit it.

Vulnerability - Buffer Overflow - Format string	Line	Is it exploitable?	
		YES	NO
[1 point] Buffer Overflow	26	☐	☒
[1 point] Format String	36	☒	☐

[2 points] 2. One of the particularly motivated students found a way to manipulate the system and obtain 6 points in a deterministic way. He noted the function **max_grade**. Provide a detailed description of the exploit that led to the execution of the function.

Note: List the vulnerabilities you are exploiting and all the exploit steps.

[0.5 points] Vulnerability to exploit: Format String on line 36.

[1.5 points] By using the **printf** on line 36, it is possible to perform a Format String attack and overwrite the **savedEIP** of the **set_notes** function. This allows the attacker to redirect execution to the **max_grade** function.

To achieve this, we need to place the target stack addresses to be overwritten onto the stack. One possible method is to inject them using environment variables. The only mitigation enabled is the stack canary; ASLR is disabled, so we can easily retrieve the necessary information:

- **WHERE_TO_WRITE:** Since ASLR is disabled, the stack position is deterministic, so we can run the binary one time to take note of the address at which the **savedEIP** is.
- **POS:** We can run the binary multiple times, use the format string to find the offset on the stack between the **printf** argument and the injected environment variables.
- **WHAT_TO_WRITE:** Since the binary is not PIE, the address of the **max_grade** function can be retrieved directly from the ELF file.

After we have collected all this information, we can build the typical format string payload.

[2 point] 3. Show the effect of one of the identified vulnerabilities on memory. If you have found at least one exploit that enables execution of the shellcode described in point 2, choose **one of them** and illustrate the **stack layout both before and after** the execution of the most relevant line for that exploit, including:

- The **direction of memory growth** (the stack grows downward) and the **relative address order** (from high to low);
- The **name of each relevant allocated variable** in the stack frame;
- The **contents of relevant registers**, such as **EBP** and **ESP**;
- The **stack frames of the current function** and any **callers**;
- The **contents of the caller's frame**, including saved return addresses and local variables.

	Before the Format String		After the Format String		
					set_notes's frame
					main's frame
					__libc_start_main's frame
Low addresses					
	Canary	<-- ESP	Canary	<-- ESP	
	savedEBP	<-- EBP	savedEBP	<-- EBP	
	savedEIP (main)		MAX_GRADE_ADDR		
	Canary		Canary		
	savedEBP		savedEBP		
	savedEIP (__libc_start_main)		savedEIP (__libc_start_main)		
					
					
	LOW=		LOW=		
	WHERE_TO_WRITE_LOW		WHERE_TO_WRITE_LOW		
	HIGH=		HIGH=		
	WHERE_TO_WRITE_HIGH		WHERE_TO_WRITE_HIGH		
High Addresses					

The stack grows from high memory addresses to low ones.

Note: This is just an example. Any approach that successfully injects the WHERE_TO_WRITE addresses onto the stack and causes execution of the max_grade function will receive full credit (2 points).

Exercise 4 - Web Vulnerabilities (6 points)

QuickCart.com is a newly launched e-commerce platform that carries a broad range of consumer goods. During its introductory period, the site is offering an extensive selection of promotional coupons that deliver significant discounts across the catalog. A dedicated support team is available 24/7 to provide prompt assistance with any customer inquiries.

```
00 @app.route('/add_coupon', methods=['POST'])
01 def add_coupon(request, session):
02     if not is_session_valid(session):
03         return redirect(url_for("login"))
04     if not session["is_admin"]:
05         return render_template("home.html", error="Forbidden"), 403
06     if "coupon_code" not in request.form.keys():
07         return render_template("home.html", error="No coupon code provided!")
08     if "discount" not in request.form.keys():
09         return render_template("home.html", error="No discount provided!")
10     coupon_code = request.form["coupon_code"]
11     discount = float(request.form["discount"])
12     coupon_q = "INSERT INTO coupons VALUES (?, ?);"
13     db_execute_query_with_statements(coupon_q, coupon_code, discount)
14     return render_template("home.html", success="Coupon added successfully!")
15
16 @app.route('/send_inquiry', methods=['GET'])
17 def send_inquiry(request, session):
18     if not is_session_valid(session):
19         return redirect(url_for("login"))
20     if "text" not in request.args.keys():
21         return render_template("home.html", error="No inquiry provided!")
22     text = request.args["text"]
23     inquiry_q = "INSERT INTO inquiries VALUES (" + user_id + ", '" + text + "');"
24     db_execute_query_with_statements(inquiry_q)
25     return render_template("home.html", success="Inquiry submitted successfully!")
26
27 @app.route('/view_inquiry', methods=['GET'])
28 def view_inquiry(request, session):
29     if not is_session_valid(session):
30         return redirect(url_for("login"))
31     if not session["is_admin"]:
32         return render_template("home.html", error="Forbidden"), 403
33     q = "SELECT * FROM inquiries ORDER BY inquiry_id LIMIT 1;"
34     result = db_execute_query(q)
35     if len(result) != 1:
36         return render_template("home.html", error="No inquiries found!")
37     text = result[0]["text"]
38     user_id = str(result[0]["user_id"])
39     return render_template("view_inquiry.html", inquiry=text, user_id=user_id)
```

```
50 <html> <!-- view_inquiry.html -->
    ...
60 <h1>Inquiry from user {{ user_id }}</h1>
61 <p>{{ inquiry }}</p>
    ...
70 </html>
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)
--	----------------------	----------------------	-------------------

Table: **coupon**

coupon_id (Primary Key) (int) (auto increment)	code (string)	discount (float)
--	------------------	---------------------

Table: **inquiries**

inquiry_id (Primary Key) (int) (auto increment)	user_id (string) (Reference: users.user_id)	text (string)
---	---	------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query** executes queries without prepared statements.
- The function **db_execute_query_with_statements** executes queries with prepared statements.
- The function **render_template** does not perform any sanitization of the input.
- The **DBMS** does not support stacked queries (e.g., "SELECT ...; INSERT INTO ...;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.

More details:

- The function **render_template** generates an HTML page from an .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL for the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided URL.
- The function **db_execute_query** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["comment"] is accessing the field "comment" of the 5th row.
- In the session, the variable **is_admin** is set to true during login only if the user is an admin.

1. [1 point] For each SQL injection vulnerability you have identified, please provide the vulnerable line and the variable(s) that can be exploited for the injection.

Line	Variable(s)
23	text

2. [1 point] For each XSS vulnerability you have identified, please provide the vulnerable variable, the XSS category, and where you would fix it. Moreover, explain why

Variable	XSS category			Lines to sanitize. It can be between two lines (e.g., 21-22) or in one line.
	Stored	Reflected	DOM-based	
text	☒	☐	☐	22-23
	☐	☐	☐	
	☐	☐	☐	

3. [1 point] For each function, specify whether they are vulnerable to CSRF attacks. Moreover, explain why they are vulnerable.

Function	Vulnerable	Not vulnerable
add_coupon	☒	☐
send_inquiry	☒	☐
view_inquiry	☐	☒

Motivation:

add_coupon and send_inquiry are both state-changing actions not protected by a CSRF token

4. [3 points] An admin received an inquiry saying there's an error on the Page QuickSmart.com/buy (whereas the actual domain of the web service is QuickCart.com). The admin clicks the link, and, after some days, the same admin notices a user buying items at a 99.99 % discount. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

It's a CSRF attack. The user sent an inquiry to an admin with a link to QuickSmart.com/buy (owned by the attacker) with several auto-submitting forms like this:

```
<form method="GET" action="www.QuickCart.com/add_coupon" onload="this.submit()">  
  <input type="hidden" name="coupon_code" value="12345">  
  <input type="hidden" name="discount" value="0.9999">  
</form>
```

Or links that are automatically clicked while visiting the page like:

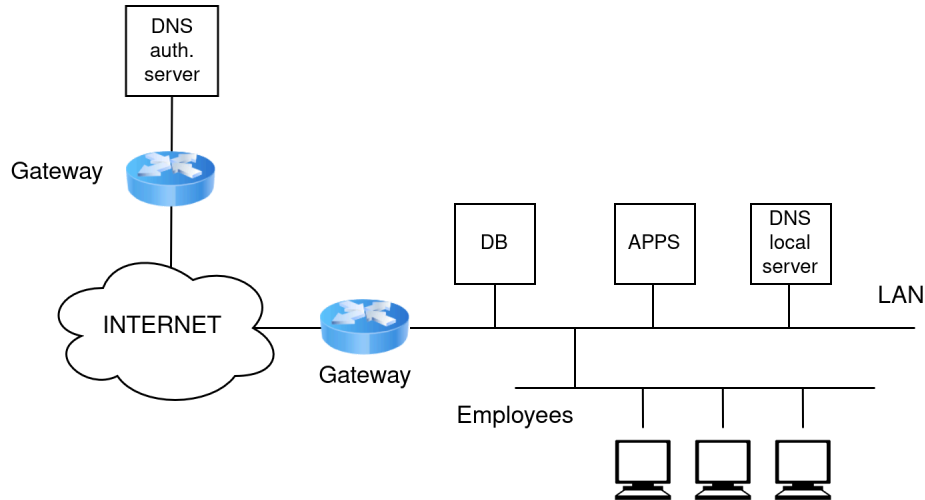
www.QuickCart.com/add_coupon?coupon_code=12345&discount=0.9999

The second is valid since it is a GET request. If it had been a POST, only the first would have been valid.

Exercise 5 - Network Security (6 points)

Jim Halpert wants to pull one of his classic pranks on Dwight Schrute. Jim overheard that Dwight needs to buy 1000 beet seeds to prepare his farm for the new season, so he wants to trick Dwight into making a false order to get 1000 pumpkin seeds. Jim already bought a proper domain “**beet_scam.com**” with IP **81.13.173.234**, set up the proper pages and forms ready to be confused for “**beet_best.com**” with IP **102.157.78.225**, the trusted site from which Dwight generally buys.

Here is a representation of the Scranton branch of the Dunder Mifflin paper company.



Communication with external sites is via **HTTPS** (we can assume that all certificates are valid and trusted); within the Dunder Mifflin LAN, all **MAC** addresses are already solved and cached, and **IP** addresses have already been assigned to each connected device; **ICMP** redirects are disabled.

[2 point] 1. What type of attack can Jim carry on? Check the name of the attack and clearly state the assumptions upon which this attack can work in the box below

☐ DoS/dDoS attack	☐ TCP spoofing attack	☐ TCP session hijacking
☐ ARP spoofing	☐ Man In The Middle	☒ DNS cache poisoning
☐ DHCP poisoning	☐ ICMP redirect	☐ No attack can be done

Since everything is on the same LAN, sniffing is possible. The attacker can sniff the ID of the DNS query made by the DNS local server to the DNS authoritative server.

The DNS protocol does not require authentication; therefore, DNS requests can be crafted.

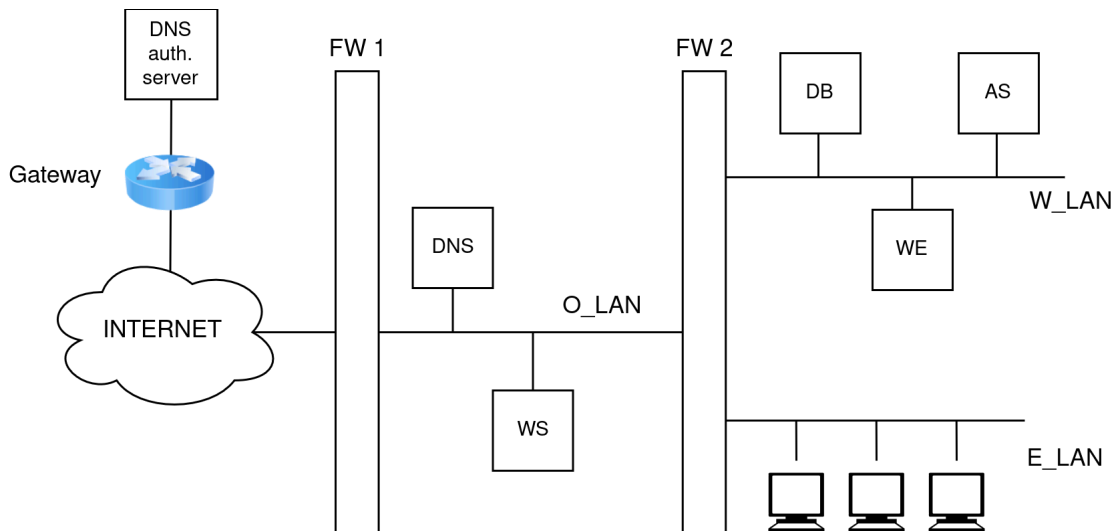
[2 points] 2. Describe how the attack works, focusing on all the required steps and the messages exchanged among all the actors involved in this scenario.

Jim asks the DNS local server to resolve for “beet_best.com”. It queries the DNS authoritative server, but Jim crafts a proper answer (he can sniff the ID of the DNS query), racing the DNS auth. Server, providing back the IP 81.13.173.234, the one from “beet_scam.com”.

When Dwight tries to connect to “beet_best.com,” the DNS resolution is cached beforehand, thus he is tricked into connecting to the wrong site.

See slides for more info

On receiving the wrong seeds, Dwight discovers that he was tricked, so he runs into Michael’s office and convinces him that something very bad is happening: they were attacked, and therefore, they need protection. After proper consultation, the network architecture of the branch is changed.



[2 point] 3. Why the previous attack cannot work anymore?

Now, employees and the DNS server are on different LANs, thus, the attacker cannot sniff the ID of the DNS query request made by the DNS local server to the DNS auth server.

Exercise 6 - Malware (6 points)

Ryuko Matoi, a fierce and rebellious high school student, is fighting back against the oppressive REVOCS Corporation. Armed with half a scissor blade and a deep grudge, she's not just taking the battle to the battlefield — she's bringing it to the digital front. To strike REVOCS where it hurts, she's developing a piece of custom malware designed to infiltrate their systems. But something's off. Ryuko reaches out to you — a trusted ally and skilled low-level hacker — with the following code snippet. She suspects the malware isn't behaving as intended, and asks for your help to review and fix it.

```
#include <stdio.h>
#include <stdint.h>
#include <x86intrin.h> // for __rdtsc()
#include <time.h>

uint64_t flag = 1;

// Handles the exception
LONG WINAPI excep_DebugBreak()
{
    flag = 0;
    Sleep(INFINITE);
    return 0;
}

unsigned __stdcall th_DebugBreak()
{
    LPTOP_LEVEL_EXCEPTION_FILTER Top =
    SetUnhandledExceptionFilter(excep_DebugBreak);
    // Raises an exception
    DebugBreak();
    SetUnhandledExceptionFilter(Top);
    return 0;
}

int main() {
    // Starts a thread running th_DebugBreak
    _beginthreadex(0, 0, th_DebugBreak, NULL, 0, 0);
    Sleep(10); // Sleep in seconds to wait thread execution
    if (flag) {
        usleep(10); // Sleep in microseconds (1us = 1e-6s)
        malicious_actions();
    }

    return 0;
}
```

Some information useful in analyzing the code:

- `__beginthreadex` starts a thread in the code
- `SetUnhandledExceptionFilter` registers an exception handler
- `DebugBreak()` raises a debugging exception
- Debuggers block by default debugging exceptions and do not pass them to the debuggee.

[4 point] 1. Please fill the following table showing the stealth techniques the malware correctly implements or tries to implement. Report the motivation only for the techniques you identify as implemented.

Technique	Is Present	Motivation (only if present)
Polymorphism	NO	[see slides for Polymorphism definition]
Metamorphism	NO	[see slides for Metamorphism definition]
Anti-Debugging	YES	The program registers a debugging exception handler and raises a debugging exception to check whether it is intercepted by the debugger or reaches the program itself.
Dormant Code	YES	There is a <code>usleep</code> call before <code>malicious_action()</code> is executed.

[2 point] 2. Please identify which are the techniques badly implemented, and propose a solution to correctly implement them.

Anti-Debugging — Incorrect Implementation:

The condition `if (flag)` is inverted. Hence, the malicious actions are executed **only if a debugger is present**. **Solution:** Invert the condition to `if (!flag)`.

Dormant Code — Ineffective Delay:

The `usleep` duration is too short to have any meaningful impact. **Solution:** Increase the sleep time to make the delay effective (usually, you can assume a sandbox to be executed for 3-5 minutes)